killMS

Release 0.1.0

Cyril Tasse

May 20, 2026

CONTENTS:

The killMS software allows for Wirtinger-based direction-dependent calibration.

CHAPTER

ONE

KILLMS

1.1 TestHarness package

1.1.1 Subpackages

TestHarness.LongAcceptanceTests package

Submodules

TestHarness.LongAcceptanceTests.TestLOFAR_J1329_p4729 module

DDFacet, a facet-based radio imaging package Copyright (C) 2013-2016 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class TestHarness.LongAcceptanceTests.TestLOFAR_J1329_p4729.**TestLOFAR_J1329_p4729**(**args*, ***kwargs*)

Bases: ClassCompareFITSImage

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

classmethod defMeanSquaredErrorLevel()

Method defining maximum tolerance for the mean squared error between any pair of FITS images. Should be overridden if another tolerance is desired Returns: constant for tolerance on mean squared error

classmethod defineImageList()

Method to define set of reference images to be tested. Can be overridden to add additional output products to the test. These must correspond to whatever is used in writing out the FITS files (eg. those in ClassDeconvMachine.py) :returns: List of image identifiers to reference and output products

classmethod defineMaxSquaredError()

Method defining maximum error tolerance between any pair of corresponding pixels in the output and corresponding reference FITS images. Should be overridden if another tolerance is desired :returns: constant for maximum tolerance used in test case setup

classmethod post_imaging_hook()

classmethod pre_imaging_hook()

classmethod timeoutsecs()

Module contents

1.1.2 Module contents

1.2 killMS package

1.2.1 Subpackages

killMS.Array package

Subpackages

killMS.Array.Dot package

Submodules

killMS.Array.Dot.NpCuda module

killMS.Array.Dot.NpDotSSE module

killMS.Array.Dot.TestDotSSE module

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

Submodules

killMS.Array.ModLinAlg module

killMS.Array.NpShared module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Array.NpShared.**DelAll**(*key=None*)

Deletes either all SharedArrays with specific key, or all sharedarrays.

Return type
None

killMS.Array.NpShared.**DelArray**(*Name*)

Wrapper for SharedArray delete classmethod.

Parameters
Name (*str*) –

Return type
None

killMS.Array.NpShared.**DicoToShared**(*Prefix*, *Dico*, *DelInput=False*)

Takes a dictionary, makes it into a SharedDict.

Parameters

- **Prefix** (*str*) –
- **Dico** (*dict*) –

Return type
dict

killMS.Array.NpShared.**GiveArray**(*Name*)

Returns the Name's SharedArray, if it exists.

Parameters
Name (*str*) –

killMS.Array.NpShared.**ListNames**()

Times SharedToDIco, then returns a list of currently-existing SharedArray names.

Return type
tuple

killMS.Array.NpShared.**PackListArray**(*Name*, *LArray*)

Packs a list of arrays into a shared dict

Parameters

- **Name** (*str*) –
- **LArray** (*tuple*) –

class killMS.Array.NpShared.**SharedDicoDescriptor**(*prefixName*, *Dico*)

Bases: object

descriptor object for SharedDico

Parameters

- **prefixName** (*str*) –
- **Dico** (*dict*) –

killMS.Array.NpShared.**SharedObjectToDico**(*SObject*)

Takes sharedobject (None or SharedDico), returns dict

Return type
dict

killMS.Array.NpShared.**SharedToDico**(*Prefix*)

Takes in SharedDico, returns dict

Parameters
Prefix (*str*) –

Return type
dict

killMS.Array.NpShared.**ToShared**(*Name*, *A*)

Takes in an array and string, constructs and outputs associated SharedArray.

Parameters

- **Name** (*str*) –
- **A** (*numpy.ndarray*) –

Return type
~SharedArray

killMS.Array.NpShared.**UnPackListArray**(*Name*)

Takes in a shared list of arrays, unpacks into a tuple

Parameters
Name (*str*) –

Return type
tuple

killMS.Array.NpShared.**zeros**(**args*, ***kwargs*)

Generates an empty SharedArray. Args, kwargs will be: name, shape, dtype=float

Return type
~SharedArray

killMS.Array.RecArrayOps module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Array.RecArrayOps.**AppendField**(*dataAll*, *FN*)

Function to add field access as attribute lookup for a tuple

Return type
numpy.record

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Data package

Submodules

killMS.Data.ClassBeam module

killMS.Data.ClassJonesDomains module

killMS.Data.ClassMS module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Data.ClassMS.**ClassMS**(*MSname*, *Col='DATA'*, *zero_flag=True*, *ReOrder=False*, *EqualizeFlag=False*, *DoPrint=True*, *DoReadData=True*, *TimeChunkSize=None*, *GetBeam=False*, *RejectAutoCorr=False*, *SelectSPW=None*, *DelStationList=None*, *Field=0*, *DDID=0*, *ReadUVWDT=False*, *ChanSlice=None*, *GD=None*, *ToRADEC=None*)

Bases: object

Class to provide relevant quantities for Measurement Set object

AddCol(*ColName*, *LikeCol='DATA'*, *ColDesc=None*, *ColDescDict=None*)

Add column to MS

Return type
None

AddUVW_dt()

Add UWV_dt info to the MS

Return type
None

CopyCol(*Colin*, *Colout*)

Copy a column from Colin to Colout

Return type
None

CopyNonSPWDependent(*MSnodata*)

Copies all MS properties which are not dependent on SPW into input MSnodata TODO: not sure how to type its output.

DelData()

Deletes weights if they are present; deletes data and flags if present

Return type
None

GiveBeam(*time*, *ra*, *dec*)

Calculates Jones beam value at provided time, ra, dec. Freq missing?

Return type
numpy.ndarray

GiveCol(*ColName*)

Open main table, return specific column, then close main table.

Return type
None

GiveDataChunk(*it0*, *it1*)

Returns data chunk between t0 and t1

Return type
dict

GiveDate(*tt*)

Gives date of input tt ETN: TODO: figure out how to say it returns a datetime object.

GiveMainTable(***kw*)

Returns main MS table, applying TaQL selection if any

Return type
table

GiveMappingAnt(*ListStrSel*, *row0row1=(None, None)*, *FlagAutoCorr=True*, *WriteAttribute=True*)

Build baseline mapping for provided ListStrSel list of baseline AntName1-AntName2 strings

Return type
numpy.ndarray

GiveUvwBL(*a0*, *a1*)

Gives uvw for requested a0-a1 baseline

Return type
numpy.ndarray

GiveVisBL(*a0*, *a1*, *col=0*, *pol=None*)

gives data for requested a0-a1 baseline

Return type
numpy.ndarray

GiveVisBLChan(*a0*, *a1*, *chan*, *pol=None*)

Gives data for requested a0-a1 baselines for requested chan, either stokes-I (pol=None) or for specific correlation (pol!=None)

Return type
numpy.ndarray

Give_dUVW_dt(*ttVec*, *A0*, *A1*, *LongitudeDeg=6.8689*, *R='UVW_dt'*)

Read or generate UVW_dt column

Return type
numpy.ndarray

LoadDDFBeam()

Loads DDF FitsBeam, ATCABeam. TODO: generalise…

Return type
None

LoadLOFAR_ANTENNA_FIELD()

Load LOFAR_ANTENNA_FIELD table of MSv2 to MS dictionary

Return type
None

LoadSR(*useElementBeam=True*, *useArrayFactor=True*)

Wrapper for lofar StationResponse which encodes SR and adds skycoord metadata to it

Return type
None

PutBackupCol(*incol='CORRECTED_DATA'*)

Add backup col for incol in MS

Return type
bool

`PutCasaCols`()
Wrapper for casacore.tables.addImagingColumns

Return type
None

`PutColInData`(*SpwChan*, *pol*, *data*)
Update self.data with provided data

Return type
None

`PutLOFARKeys`()
Adds LOFAR antenna table elements to ClassMS dict

Return type
None

`PutNewCol`(*Name*, *LikeCol='CORRECTED_DATA'*)
Add new column called Name in MS, based on LikeCol

Return type
None

`ReadData`(*t0=0*, *t1=-1*, *DoPrint=False*, *ReadWeight=False*)
Actual data reader function

Return type
bool

`ReadMSInfo`(*MSname*, *DoPrint=True*)
Reads all MS info, initialises ClassMS properties, optionally rephases to target coords

Return type
None

`Restore`()
Copy backup CORRECTED_DATA, FLAG to current CORRECTED_DATA,FLAG.

Return type
None

`Rotate`(*DATA*, *RotateType=['uvw', 'vis']*, *Sense='ToTarget'*, *DataFieldName='data'*)
rephase, apply to RotateType array, Sense says which way we go, and apply to DataFIeldName columns

Return type
None

`RotateMS`(*radec*)
Rephase MS to requested radec, update relevant metadata

Return type
None

`SaveAllDataStruct`()
Writes in-memory A0, A1, TIME, TIME_CENTROID, UVW and FLAG to MS

Return type
None

SaveVis(*vis=None*, *Col='CORRECTED_DATA'*, *spw=0*, *DoPrint=True*)

Write provided vis datacolumn, or self.data, into requested Col

Return type
None

ToOrigFreqOrder(*data*)

Reverts channels to their original order, in case of reorder

Return type
numpy.ndarray

ZeroFlagSave(*spw=0*)

Sets all flags to 0

Return type
None

plotBL(*a0*, *a1*, *pol=0*)

Plot specific baseline

Return type
None

radec2lm_scalar(*ra*, *dec*, *original=False*)

Calculates scalar l,m from ra dec and ms phase centre

Return type
tuple

killMS.Data.ClassReCluster module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Data.ClassReCluster.**ClassReCluster**(*GD*)

Bases: object

docstring

ReClusterSkyModel(*SM*, *MSName*)

Change clustering on a given SkyModel (SM) to match that of a reference SolRefFile MSName is only read to initialise SolRefFile if PreApplySols are not provided in terminal

Return type
None

killMS.Data.ClassVisServer module

killMS.Data.ClassWeighting module

killMS.Data.sidereal module

class killMS.Data.sidereal.**AltAz**(*alt*, *az*)

Bases: object

Represents a sky location in horizon coords. (altitude/azimuth)

Exports/Invariants:
.alt: [altitude in radians, in [-pi,+pi]] .az: [azimuth in radians, in [0,2*pi]]

raDec(*lst*, *latLon*)

Convert horizon coordinates to equatorial.

[(lst is a local sidereal time as a SiderealTime instance) and

(latLon is the observer's position as a LatLon instance) ->
return the corresponding equatorial coordinates as a RADec instance]

class killMS.Data.sidereal.**FixedZone**(*hh*, *mm*, *name*)

Bases: tzinfo

Represents a time zone with a fixed offset east of UTC.

Exports:

FixedZone (hours, minutes, name):

[(hours is a signed offset in hours as an int) and

(minutes is a signed offset in minutes as an int) ->
return a new FixedZone instance representing those offsets east of UTC]

State/Invariants:

.__offset:

[a datetime.timedelta representing self's offset
east of UTC]

.__name:
[as passed to the constructor's name argument]

dst(*dt*)

Return self's daylight time offset.

tzname(*dt*)

Return self's name.

Return type
str

utcoffset(*dt*)

Return self's offset east of UTC.

class killMS.Data.sidereal.**JulianDate**(*j*, *f=0.0*)

Bases: object

Class to represent Julian-date timestamps.

State/Invariants:
.f: [(Julian date as a float) - JULIAN_BIAS]

`datetime`()

Convert to a standard Python datetime object in UT.

`static fromDatetime`(*dt*)

Create a JulianDate instance from a datetime.datetime.

[dt is a datetime.datetime instance ->

if dt is naive ->
return the equivalent new JulianDate instance, assuming dt expresses UTC

else ->
return a new JulianDate instance for the UTC time equivalent to dt]

`offset`(*delta*)

Return a new JulianDate for self+(delta days)

[delta is a number of days as a float ->
return a new JulianDate (delta) days in the future, or past if negative]

`class killMS.Data.sidereal.LatLon`(*lat*, *lon*)

Bases: `object`

Represents a latitude+longitude.

`class killMS.Data.sidereal.MixedUnits`(*factors*)

Bases: `object`

Represents a system with mixed units, e.g., hours/minutes/seconds

`format`(*coeffs*, *decimals=0*, *lz=False*)

Format mixed units.

[(coeffs is a sequence of numbers as returned by
MixedUnits.singleToMix()) and (decimals is a nonnegative integer) and (lz is a bool) ->

return a list of strings corresponding to the values of coeffs, with all the values but the last formatted as integers, all values zero padded iff lz is true, and the last value with (decimals) digits after the decimal point]

`mixToSingle`(*coeffs*)

Convert mixed units to a single value.

[coeffs is a sequence of numbers not longer than

len(self.factors)+1 ->
return the equivalent single value in self's system]

`singleToMix`(*value*)

Convert to mixed units.

[value is a float ->
return value as a sequence of coefficients in self's system]

`class killMS.Data.sidereal.RADec`(*ra*, *dec*)

Bases: `object`

Represents a celestial location in equatorial coordinates.

Exports/Invariants:
.ra: [right ascension in radians] .dec: [declination in radians]

`altAz`(*h*, *lat*)

Convert equatorial to horizon coordinates.

[(h is an object's hour angle in radians) and

(lat is the observer's latitude in radians) ->
return self's position in the observer's sky in horizon coordinates as an AltAz instance]

`hourAngle`(*utc*, *eLong*)

Find the hour angle at a given observer's location.

[(utc is a Universal Time as a datetime.datetime) and

(eLong is an east longitude in radians) ->
return the hour angle of self at that time and longitude, in radians]

`class killMS.Data.sidereal.SiderealTime`(*hours*)

Bases: `object`

Represents a sidereal time value.

State/Internals:
.hours: [self as 15-degree hours] .radians: [self as radians]

`SIDEREAL_C = 1.002738`

`static factorB`(*yyyy*)

Compute sidereal conversion factor B for a given year.

[yyyy is a year number as an int ->
return the GST at time yyyy-01-00T00:00]

`static fromDatetime`(*dt*)

Convert civil time to Greenwich Sidereal.

[dt is a datetime.datetime instance ->

if dt has time zone information ->
return the GST at the UTC equivalent to dt

else ->
return the GST assuming dt is UTC]

`gst`(*eLong*)

Convert LST to GST.

[self is local sidereal time at longitude eLong

radians east of Greenwich ->
return the equivalent GST as a SiderealTime instance]

`lst`(*eLong*)

Convert GST to LST.

[(self is Greenwich sidereal time) and

(eLong is a longitude east of Greenwich in radians) ->
return a new SiderealTime representing the LST at longitude eLong]

utc(*date*)

Convert GST to UTC.

[date is a UTC date as a datetime.date instance ->
return the first or only time at which self's GST occurs at longitude 0]

class killMS.Data.sidereal.**USTimeZone**(*hh*, *mm*, *name*, *stdName*, *dstName*)

Bases: tzinfo

Represents a U.S. time zone, with automatic daylight time.

Exports:

USTimeZone (hh, mm, name, stdName, dstName):

[(hh is an offset east of UTC in hours) and
(mm is an offset east of UTC in minutes) and (name is the composite zone name) and (stdName is the non-DST name) and (dstName is the DST name) ->

return a new USTimeZone instance with those values]

State/Invariants:

.__offset:
[self's offset east of UTC as a datetime.timedelta]

.__name: [as passed to constructor's name] .__stdName: [as passed to constructor's stdName] .__dstName: [as passed to constructor's dstName]

DST_END_2007 = datetime.datetime(1, 11, 1, 2, 0)

DST_END_OLD = datetime.datetime(1, 10, 25, 2, 0)

DST_START_2007 = datetime.datetime(1, 3, 8, 2, 0)

DST_START_OLD = datetime.datetime(1, 4, 1, 2, 0)

dst(*dt*)

Return the current DST offset.

[dt is a datetime.date ->
if daylight time is in effect in self's zone on date dt ->

return +1 hour as a datetime.timedelta

else ->
return 0 as a datetime.delta]

tzname(*dt*)

datetime -> string name of time zone.

utcoffset(*dt*)

datetime -> timedelta showing offset from UTC, negative values indicating West of UTC

killMS.Data.sidereal.**coordRotate**(*x*, *y*, *z*)

Used to convert between equatorial and horizon coordinates.

[x, y, and z are angles in radians ->
return (xt, yt) where xt=arcsin(sin(x)*sin(y)+cos(x)*cos(y)*cos(z)) and yt=arccos((sin(x)-sin(y)*sin(xt))/(cos(y)*cos(xt)))]

killMS.Data.sidereal.**dayNo**(*dt*)

Compute the day number within the year.

[dt is a date as a datetime.datetime or datetime.date ->
return the number of days between dt and Dec. 31 of the preceding year]

Return type
int

killMS.Data.sidereal.**firstSundayOnOrAfter**(*dt*)

Find the first Sunday on or after a given date.

[dt is a datetime.date ->
return a datetime.date representing the first Sunday on or after dt]

Return type
timedelta

killMS.Data.sidereal.**hourAngleToRA**(*h*, *ut*, *eLong*)

Convert hour angle to right ascension.

[(h is an hour angle in radians as a float) and
(ut is a timestamp as a datetime.datetime instance) and (eLong is an east longitude in radians) ->

return the right ascension in radians corresponding to that hour angle at that time and location]

Return type
float

killMS.Data.sidereal.**hoursToRadians**(*hours*)

Convert hours (15 degrees) to radians.

Return type
float

killMS.Data.sidereal.**parseAngle**(*s*)

Validate and convert an external angle.

[s is a string ->

if s is a valid external angle ->
return s in radians

else -> raise SyntaxError]

killMS.Data.sidereal.**parseDate**(*s*)

Validate and convert a date in external form.

[s is a string ->

if s is a valid external date string ->
return that date as a datetime.date instance

else -> raise SyntaxError]

Return type
date

killMS.Data.sidereal.**parseDatetime**(*s*)

Parse a date with optional time.

[s is a string ->

if s is a valid date with optional time ->
return that timestamp as a datetime.datetime instance

else -> raise SyntaxError]

Return type
datetime

killMS.Data.sidereal.**parseFixedZone**(*s*)

Convert a +hhmm or -hhmm zone suffix.

[s is a string ->

if s is a time zone suffix of the form "+hhmm" or "-hhmm" ->
return that zone information as an instance of a class that inherits from datetime.tzinfo

else -> raise SyntaxError]

killMS.Data.sidereal.**parseFloat**(*s*, *message*)

Parse a floating-point number at the front of s.

[(s is a string) and

(message is a string describing what is expected) ->

if s begins with a floating-point number ->
return (x, tail) where x is the number as type float and tail is the part of s after the match

else -> raise SyntaxError, "Expecting (message)"]

killMS.Data.sidereal.**parseFloatSuffix**(*s*, *codeRe*, *message*)

Parse a float followed by a letter code.

[(s is a string) and
(codeRe is a compiled regular expression) and (message is a string describing what is expected) ->

if s starts with a float, followed by code (using case-insensitive comparison) ->

return (x, tail) where x is that float as type float and tail is the part of s after the float and code

else -> raise SyntaxError, "Expecting (message)"]

killMS.Data.sidereal.**parseHours**(*s*)

Validate and convert a quantity in hours.

[s is a non-empty string ->

if s is a valid mixed hours expression ->
return the value of s as decimal hours

else -> raise SyntaxError]

killMS.Data.sidereal.**parseLat**(*s*)

Validate and convert an external latitude.

[s is a nonempty string ->

if s is a valid external latitude ->
return that latitude in radians

else -> raise SyntaxError]

killMS.Data.sidereal.**parseLon**(*s*)

Validate and convert an external longitude.

[s is a nonempty string ->

if s is a valid external longitude ->
return that longitude in radians

else -> raise SyntaxError]

killMS.Data.sidereal.**parseRe**(*s*, *regex*, *message*)

Parse a regular expression at the head of a string.

[(s is a string) and
(regex is a compiled regular expression) and (message is a string describing what is expected) ->

if s starts with a string that matches regex ->
return (head, tail) where head is the part of s that matched and tail is the rest

else ->
raise SyntaxError, "Expecting (message)"]

killMS.Data.sidereal.**parseTime**(*s*)

Validate and convert a time and optional zone.

[s is a string ->

if s is a valid time with optional zone suffix ->
return that time as a datetime.time

else -> raise SyntaxError]

Return type
time

killMS.Data.sidereal.**parseZone**(*s*)

Validate and convert a time zone suffix.

[s is a string ->

if s is a valid time zone suffix ->
return that zone's information as an instance of a class that inherits from datetime.tzinfo

else -> raise SyntaxError]

killMS.Data.sidereal.**raToHourAngle**(*ra*, *ut*, *eLong*)

Convert right ascension to hour angle.

[(ra is a right ascension in radians as a float) and
(ut is a timestamp as a datetime.datetime instance) and (eLong is an east longitude in radians) ->

return the hour angle in radians at that time and location corresponding to that right ascension]

Return type
float

killMS.Data.sidereal.**radiansToHours**(*radians*)

Convert radians to hours (15 degrees).

Return type
float

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Gridder package

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other package

Submodules

killMS.Other.ClassClip module

killMS.Other.ClassFitAmp module

killMS.Other.ClassFitTEC module

killMS.Other.ClassPrint module

class killMS.Other.ClassPrint.**ClassPrint**(*HW=20*, *quote=''*)

Bases: object

Class that defines the terminal print parameters.

Parameters

- **HW** (*int*) –
- **quote** (*str*) –

Print(*par*, *value*, *value2=None*)

Print function to align logger outputs.

Parameters

- **self** – ClassPrint
- **par** (*str*) – Parameter to print value of
- **value** (*str*) – Value of the parameter
- **value2** – Optional second parameter value

Return type
None

Print2(*par*, *value*, *helpit=''*, *col='white'*)

Print function to align logger outputs, with colourful formatting. Prints out pretty logger.

Parameters

- **self** – ClassPrint
- **par** (*str*) – parameter to print value of
- **value** (*str*) – Value of the parameter
- **helpit** (*str*) – Help string to be printed. Default is no help given.
- **col** (*str*) – Colour of the print. Default is white.

Return type
None

getWidth()

Function to retrive current terminal size. Returns width.

param self: ClassPrint

Return type
int

killMS.Other.ClassTimeIt module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Other.ClassTimeIt.**ClassTimeIt**(*name=''*, *f=1.0*)

Bases: object

Timing class for logging purposes.

AddDt(*Var*)

Finds current time, finds dt from t0, updates t0 to current time, and adds dt value to the input float.

Return type
float

disable()

Setter to disable the class

Return type
None

enableIncr(*incr=1*)

Function to enable incrementation. Initialises Counter to 0. :param incr: not used :type incr: int

Return type
None

reinit()

Reinitiatlise the timer

Return type
None

timeit(*stri=' Time'*, *hms=False*)

Time an event. :param stri: optional prefix :type stri: str :param hms: reformat time to HMS :type hms: bool

Return type
float

timeitHMS(*stri=' Time'*)

Updates self.t0 to current time. :param stri: fofmating string, not used here. :type stri:str

Return type
None

killMS.Other.Counter module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

`class killMS.Other.Counter.Counter(N=1)`

Bases: `object`

Counter class

Parameters
N (*int*) –

`class killMS.Other.Counter.CounterTime(dt=60)`

Bases: `object`

Count for time, in batches of 60.

Parameters
dt (*int*) –

killMS.Other.ModChanEquidistant module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

`killMS.Other.ModChanEquidistant.IsChanEquidistant(ChanFreq)`

Function to determine if dataset channels are equidistant. TODO: check type of ChanFreq.

Parameters
ChanFreq – Channel frequencies array.

Return type
int

killMS.Other.ModColor module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.ModColor.**Sep**(*strin=None*, *D=1*)

Returns a separator string, optionally annotated by the provided strin descriptor.

Parameters

- **strin** – Input value to be formatted
- **D** (*int*) – add input string as a Descriptor (?) for the separator

Return type
str

killMS.Other.ModColor.**Str**(*strin0*, *col='red'*, *Bold=True*)

Function to format strings for terminal log. Returns formatted string.

Parameters

- **strin0** – Input value to format.
- **col** (*str*) – string colour
- **Bold** (*bool*) – Description

Return type
str

killMS.Other.ModColor.**Title**(*strin*, *Big=False*)

Prints the Title for a given section. If Big is true, adds additional separator lines at the top and bottom of the Title.

Parameters

- **strin** – Title descriptor string.
- **Big** (*bool*) – Option to add extra sepatarators at top and bottom.

Return type
None

killMS.Other.ModColor.**disable**()

Function to set all logger environment variables to be empty: disables colourful logs.

Return type
None

killMS.Other.ModParsetType module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.ModParsetType.**DictToParset**(*Dict*, *fout*)

Function to write Dict to Parset.

Parameters

- **Dict** – Dictionary to write out
- **fout** (*str*) – Filename to write Dict to

Return type
None

killMS.Other.ModParsetType.**ParsetToDict**(*fname*)

Function to convert parset to dictionary. Returns Dict

Parameters
fname (*str*) – Filename to read from

Return type
dict

killMS.Other.MyPickle module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.MyPickle.**Load**(*filein*)

Custom pickle Load function. Returns object and values. TODO: how to type this? is it always a dict?

Parameters
filein (*str*) – filename to be loaded

Return type
dict

killMS.Other.MyPickle.**Save**(*Obj*, *fileout*)

Custom pickle Save function

Parameters

- **Obj** – Object to be pickled
- **fileout** (*str*) – Pickle filename

Return type
None

killMS.Other.PrintOptParse module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.PrintOptParse.**PrintOptParse**(*Obj*, *ValObj*, *RejectGroup=[]*)

Function to print OptParse dictionary values.

Parameters

- **Obj** – Object, first output of MyPickle invocation
- **ValObj** – Object values, second output of MyPickle invocation
- **RejectGroup** – Array including titles to skip the print of.

Return type
None

killMS.Other.PrintOptParse.**test**()

Unit test function for MyOptParse. TODO: Harmonise with other unit tests as pytest module.

Return type
None

killMS.Other.PrintOptParse.**test2**()

Second unit test for MyOptParse. Tests whether CohJones gets rejected when KAFCA is invoked?

Return type
None

killMS.Other.findrms module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.findrms.**findrms**(*mIn*, *maskSup=1e-07*)

Function to estimate the root-mean-square noise of an mIn array(?) Returns rms value.

Parameters

- **mIn** – Input array.
- **maskSup** (*float*) – threshold above which values are ignored.

Return type
float

killMS.Other.least_squares module

killMS.Other.logo module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.logo.**print_logo**()

Prints a cute killMS ASCII art block, with version.

Return type
None

killMS.Other.logo.**report_version**()

Function to find and return the current version of killMS.

Return type
str

killMS.Other.rad2hmsdms module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.rad2hmsdms.**rad2hmsdms**(*rad*, *Type='dec'*, *deg=False*)

Converts radians to SkyCoord.to_string('hmsdms') equivalent. Deprecated, should be replaced with astropy function and units. Returns HMS string.

Parameters

- **rad** (*float*) – Input angle in radians.
- **Type** (*str*) – Checks if declination or hour angle (RA). Converts properly.
- **deg** (*bool*) – Option to take in deg and convert to rad.

Return type

str

killMS.Other.reformat module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Other.reformat.**reformat**(*ssin*, *slash=True*, *LastSlash=True*)

Function that takes in a string and returns it reformatted. Formatting involves: keeping prefix and suffix slashes (slash=True)

keeping only suffix slash (LastSlash=True) if both are true, prefix and suffix are kept

Parameters

- **ssin** (*str*) – Input string
- **slash** (*bool*) – do we keep prefix and suffix slaches
- **LastSlash** (*bool*) – do we keep suffix slash

Return type
str

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Parset package

Submodules

killMS.Parset.ClassPrint module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Parset.ClassPrint.**ClassPrint**(*HW=20*, *quote=''*)

Bases: object

Class that defines the terminal print parameters.

Parameters

- **HW** (*int*) –
- **quote** (*str*) –

Print(*par*, *value*, *value2=None*)

Print function to align logger outputs.

Parameters

- **self** – ClassPrint
- **par** (*str*) – Parameter to print value of

- **value** (*str*) – Value of the parameter
- **value2** – Optional second parameter value

Return type
None

Print2(*par*, *value*, *helpit=''*, *col='white'*)

Print function to align logger outputs, with colourful formatting. Prints out pretty logger.

Parameters

- **self** – ClassPrint
- **par** (*str*) – parameter to print value of
- **value** (*str*) – Value of the parameter
- **helpit** (*str*) – Help string to be printed. Default is no help given.
- **col** (*str*) – Colour of the print. Default is white.

Return type
None

getWidth()

Function to retrive current terminal size. Returns width.

param self: ClassPrint

Return type
int

killMS.Parset.MyOptParse module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Parset.MyOptParse.**MyOptParse**(*usage='Usage: %prog --ms=somename.MS <options>'*, *version='v2.7-235-g66a31d5'*, *description='Questions and suggestions: cyril.tasse@obspm.fr'*, *DefaultDict=None*)

Bases: object

Customised killMS wrapper for OptParse.

Finalise()

Wrapper to opt.add_option_group Invoked once the CurrentGroup is fully initialised.

Parameters
self – MyOptParse

Return type
None

`GiveDicoConfig`()

Function to build the full DefaultDict out of self.options

Parameters
self – MyOptParse

Return type
dict

`GiveKeyDest`(*GroupKey*, *Name*)

Automated "Groupkey_Name" key builder for OptParse

Parameters

- **self** – MyOptParse
- **GroupKey** – Key of the group being invoked
- **Name** – Name of the option

Return type
str

`GiveKeysOut`(*KeyDest*)

Split a KeyDest into GroupKey, Name

Parameters

- **self** – MyOptParse
- **KeyDest** – Groupkey_Name key string

Return type
str

`GiveOptionObject`()

TODO: reads, sets and returns the Options for a given Object

Parameters
self – MyOptParse

Return type
object

`OptionGroup`(*Name*, *key=None*)

Function which either finalises or initialises a DicoGroupDesc

Parameters

- **self** – MyOptParse
- **Name** – Name of the OptionGroup
- **key** – optionally, set CurrentGroupKey

Return type
None

`Print`(*RejectGroup=[]*)

Function to print current options to terminal

Parameters

- **self** – MyOptParse
- **RejectGroup** – Array of groups to skip when printing

Return type
None

ReadInput()

Wrapper for opt.parse_args() sets self.options and self.arguments invokes GiveDicoConfig function, which it sets as the DefaultDict

Parameters
self – MyOptParse

Return type
None

ToParset(*ParsetName*)

Function to write current options to parset file

Parameters

- **self** – MyOptParse
- **ParsetName** – Filename for the parset in which to write current options

Return type
None

add_option(*Name='Mode'*, *help='Default %default'*, *type='str'*, *default=None*)

Wrapper for optparse add_option. Invokes the DefaultDict if provided and no default given in this invocation. Useful for batch defaulting.

Parameters

- **self** – MyOptParse
- **Name** – Name of the option
- **help** – Help message for the option
- **type** – Type of the option's value
- **default** – Default value for the option

Return type
None

killMS.Parset.MyOptParse.**test**()

Unit test for MyOptParse. TODO: harmonise with other pytest modules.

killMS.Parset.PrintOptParse module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Parset.PrintOptParse.**PrintOptParse**(*Obj*, *ValObj*, *RejectGroup=[]*)

Function to print OptParse dictionary values.

Parameters

- **Obj** – Object, first output of MyPickle invocation
- **ValObj** – Object values, second output of MyPickle invocation
- **RejectGroup** – Array including titles to skip the print of.

Return type
None

killMS.Parset.PrintOptParse.**test**()

Unit test function for MyOptParse. TODO: Harmonise with other unit tests as pytest module.

killMS.Parset.PrintOptParse.**test2**()

Second unit test for MyOptParse. Tests whether CohJones gets rejected when KAFCA is invoked?

Return type
None

killMS.Parset.ReadCFG module

ReadCFG contains functions to read parset.cfg binary files and generate a Parset object. It also includes reformating functions to ensure compatibility with internal code.

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Parset.ReadCFG.**FormatDico**(*DicoIn*)

Wrapper for FormatValue function, takes in a dicomodel and returns an ordered dicomodel. returns OrderedDict dico with either DicoIn values, or None values.

Parameters
DicoIn – DicoModel object

Return type
dict

killMS.Parset.ReadCFG.**FormatValue**(*ValueIn*, *StrMode=False*)

Function to reformat input values.

Parameters

- **ValueIn** – Value to be reformated

- **StrMode** – boolean. If True, do not reformat the input.

class killMS.Parset.ReadCFG.**Parset**(*File='../Parset/DefaultParset.cfg'*)

Bases: object

Class that defines the Parset object

ConfigSectionMap(*section*)

used by Read to do section-by-section initialisation of the param OrderedDicts

Parameters

- **self** – Description
- **section** – Description

Return type
dict

Read()

Reads a config file. Depends on configparser and ConfigSectionMap. Creates the following self objects: self.Config self.DicoPars

Parameters
self – Parset object

Return type
None

update(*other*)

Updates this Parset object with all keys and values found in provided parset object

Parameters

- **self** – Parset object
- **other** – Parset object

Return type
None

killMS.Parset.ReadCFG.**test**()

Function to test whether parset class loads correctly with default values.

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Plot package

Submodules

killMS.Plot.GiveNXNYPanels module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Plot.GiveNXNYPanels.**GiveNXNYPanels**(*Ns*, *ratio=1.6*)

Function that splits NS into NX,NY values needed to fit the requested ratio

Return type
(<class 'int'>, <class 'int'>)

killMS.Plot.GiveRectSubplot module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Plot.GiveRectSubplot.**giveRectSubPlot**(*nx*, *ny*, *fig=None*, *pos=(0.1, 0.1, 0.9, 0.9)*, *xylabel=(False, False)*)

Bases: object

giveRect(*ii*)

Returns positions and widths of the rectangle associated with subplot ii

Return type
tuple

give_axis(*i*)

Returns an Axes object added to self.fig, with desired tick behaviour defined here.

killMS.Plot.Graph module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Plot.Graph.**ClassMplWidget**(*Ntot*, *nx=None*, *ny=None*, *TBar=True*, *NPlots=None*, *SaveName='Fig'*, ***kwargs*)

Bases: object

Define self.mplwidget, with specific subplot, axes properties, patch facecolor defines self.CoordMachine as a giveRectSubPlot return, set ShowTicks to True

class ClassStrValPixel(*Dico*)

Bases: object

Get string-formated value of individual pixel

StrValPixel(*x*, *y*)

Returns string to print value of data at x,y

Return type
str

toPix(*x0*, *x1*, *xi*, *N*)

Returns relative position of xi along x0-x1 as an int

Return type
int

addToolBar()

define self.navi_toolbar, add it to self.gridLayout_2

Return type
None

cla()

update canvas, update currentIIm and currentIPlot

Return type
None

draw()

update canvas, flush_events

Return type
None

imshow(**args*, ***kwargs*)

if no args provided, set them to a 200x200 random field if self.currentIm set to -1, initialise empty array in self.DicoAxis, and populate it as though it was previously present; then, pylab.draw() if data==None

specifically, set it to 200x200 randn, then update self.DicoAxis if currentIm is the last im, set it to -, reiterate ThisPlot.set_data, self.update

Return type
None

plot(**args*, ***kwargs*)

if args are empty, set x,y to [0],[0]; yminx,ymax,xmin,xmax to -3,3, args to (x,y) otherwise, read x,y from args[0,1] read xmin,xmax, ymin,ymax from x,y; set limts from minmax if the currentIAx NLines is 0 or currentIPlot > NLines, iterate NLines, assing currentIAx x,y,lims. account for empty dictionary fields end with the append of a plot to self.DicoAxis

Return type
None

savefig()

savefig to self.SaveName_self.CounterSave.png

Return type
None

setSaveName(*name*)

Define self.SaveName variable

Return type
None

set_clim(*axis*, *vmin*, *vmax*)

read vmin vmax from thisplot im, set_clim to it, write lims to self.dixoaxis, update plot

Return type
None

set_extent(*axis*, *extent*)

set_extent input extent to ThisPlot on input axis

Return type
None

set_title(*txt*)

set title on dicoaxis currentiax ax, then draw

Return type
None

set_xylims(*xlim*, *ylim*)

Set xlim, ylim on self.DicoAxis[self.CurrentIAx][“ax”], then draw

Return type
None

subplot(*iAx*, *share=-1*, *draw=True*, ***kwargs*)

check for iAx in self.DicoAxis.keys(); add it if not yet present set currentiAx, currentAx, Im, IPlot

Return type
None

text(**args*, ***kwargs*)

if args is not an array, set x,y to 0 and txt to “” otherwise, read x,y,txt from args 0,1,2 if text is not already in self.DicoAxis[self.CurrentIAx].keys: add it ThisPlot = self.DicoAxis…[“Text”] self.update(ThisPlot)

Return type
None

update(*ThisPlot*)

redraw CurrentAx

Return type
None

updateThread(*ThisPlot*, *data*)

idk where AThread is defined. This sets and starts it.

Return type
None

updateThread2(*ThisPlot*, *data*)

idk where GenericThread is defined. looks like it dispatches the artist updates in parallel

Return type
None

killMS.Plot.Graph.**test**()

Functional test for the plotter

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Predict package

Submodules

killMS.Predict.ClassImageSM2 module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Predict.ClassImageSM2.**ClassImageSM**

Bases: object

initialised to read self.Type.

class killMS.Predict.ClassImageSM2.**ClassPreparePredict**(**args*, ***kwargs*)

Bases: ClassImagerDeconv

Inherits from DDFacet.Imager.ClassDeconvMachine import ClassImagerDeconv

LoadModel()

Start by initialising the ClassModModelMachine from the globaldict build self.MM as GiveInitialisedMMFFromFile on the FileDicoModel if asked to filter negative components, do so if MaskImage is provided, initialise CleanMaskedComponents get model_freqs from VS.FreqBandChannelsDegrid get ModelImage from self.MM.GiveModelImage set self.FacetMachine.ToCasaImage set NFacets read NodesCat build ClusterCat. TODO: ETN: GENERALISE THIS IN THE SCHEMA build catalog TODO:ETN: BIS per-facet, read l,m, build distance grid per-facet, get self.FacetMachine.DicoImager from idDir set self.SM.ClusterCat, self.SM>SourceCat import APP from DDF multiprocess startworkers FacetMachine.initCFInBackground self.FacetMachine.awaitInitCompletion initialise class variables from the above

Return type
None

PrepareGridMachinesMapping()

build soldir - gridmachine mapping read ClusterCat per clustercat dir, set DicoJonesDirToFacet read model_dict from self.FacetMachine._model_dict; reload it per-facet, get iDirJones, sumflux, sumI if the iFacet sumflux is not zero, tabulate FacetsID and sumflux per jonesdir (tessel) per-facet, read the grid, set the DicoImager and DIcoJonesDirToFacet copy ClusterCat, NDirs to build new ones (in case of empty facets within tessels) this class imports the beam from killMS.Data read solve threshold from ImageSkyModel. get apparent flux max per jones-dir. if this appflux is less than overall max appflux * threshold, remove it. reset mappings from original clustercat to kept one reset NDirs etc return True

Return type
True

class killMS.Predict.ClassImageSM2.**Worker**(*work_queue*, *result_queue*, *argsImToGrid=None*, *IdSharedMem=None*, *DicoImager=None*, *FacetMode='Fader'*, *GD=None*)

Bases: Process

Initialise class variables, including shared-mem images

run()

Parallel run. Read iFacet, FacetDataCache, image. read SPHe sharedarray read spatial weights from sharedmem initialise imtogrid calc grid, sumflux via im2grid.givemodeltessel push modelgrid to sharedmem add success to result_queue

shutdown()

killMS.Predict.PredictGaussPoints_NumExpr module

killMS.Predict.PredictGaussPoints_NumExpr5 module

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Simul package

Submodules

killMS.Simul.DoSimul module

killMS.Simul.MakeClusterCat module

killMS.Simul.MakeModelImage module

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.Weights package

Submodules

killMS.Weights.W_AntFull module

killMS.Weights.W_DiagBL module

class killMS.Weights.W_DiagBL.**ClassCovMat**(***kwargs*)

Bases: object

Finalise()

giveWeigth()

giveWeigthChunk(*A0*, *A1*, *DATA*)

giveWeigthParallel()

killMS.Weights.W_Imag module

killMS.Weights.W_ImagCov module

killMS.Weights.W_TimeCov module

Module contents

killMS.Wirtinger package

Submodules

killMS.Wirtinger.ClassAverageMachine module

killMS.Wirtinger.ClassEvolve module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.Wirtinger.ClassEvolve.**ClassModelEvolution**(*iAnt*, *iChanSol*, *WeightType='exp'*, *WeigthScale=1*, *order=1*, *StepStart=5*, *BufferNPoints=10*, *sigQ=0.01*, *DoEvolve=True*, *IdSharedMem=''*)

Bases: object

Evolve(*xEst*, *Pa*, *CurrentTime*)

updates G, Pa for CurrentTime. xEst not used... should remove applies exponential weighting to surrounding estimates for real and imag components

Return type
(numpy.ndarray, numpy.ndarray)

Evolve0(*Gin*, *Pa*, *kapa=1.0*)

Return type
numpy.ndarray

killMS.Wirtinger.ClassJacobianAntenna module

killMS.Wirtinger.ClassSolPredictMachine module

class killMS.Wirtinger.ClassSolPredictMachine.**ClassSolPredictMachine**(*GD*)

Bases: object

Class which contains GiveClosestSol function. Initialised with GlobalDict. Internal vars: self.DicoSols, self.Sols (twice) self.FreqDomains self.MeanFreqDomains self.t0 self.t1 self.tmean self.G self.ClusterCat (twice) self.raSol, self.decSol

GiveClosestSol(*t*, *freq_domains*, *ants*, *ras*, *decs*)

Finds indices of the nearest solutions in RA, Dec, freq, time, and pol for the current chunk. Returns the self.G values for these nearest coordinates.

Return type
numpy.ndarray

killMS.Wirtinger.ClassSolPredictMachine.**D**(*a0*, *a1*, *b0=None*, *b1=None*)

a0, b0 are RA, Dec. a1, b1 are the solution RA, Dec. This function calculates a distance; idk why it is RMS distance.

Return type
numpy.ndarray

killMS.Wirtinger.ClassSolverEKF module

killMS.Wirtinger.ClassSolverLM module

killMS.Wirtinger.ClassWirtingerSolver module

Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.cbuild package

Subpackages

killMS.cbuild.Array package

Subpackages

killMS.cbuild.Array.Dot package

Submodules

killMS.cbuild.Array.Dot.dotSSE module

Module contents

Module contents

killMS.cbuild.Gridder package

Module contents

killMS.cbuild.Predict package

Submodules

killMS.cbuild.Predict.predict3x module

Module contents

Submodules

killMS.cbuild.FindNumpyPath module

Module contents

1.2.2 Submodules

1.2.3 killMS.AQWeight module

1.2.4 killMS.BLCal module

1.2.5 killMS.ClipCal module

class killMS.ClipCal.**ClassClipMachine**(*MSName*, *Th=20.0*, *ColName='CORRECTED_DATA'*, *SubCol=None*, *WeightCol='IMAGING_WEIGHT'*, *ReinitWeights=False*, *InFlagCol='FLAG'*)

Bases: object

ClipWeights()

killMS.ClipCal.**driver**()

killMS.ClipCal.**main**(*options=None*)

killMS.ClipCal.**read_options**()

1.2.6 killMS.InterpSols module

1.2.7 killMS.MakePlotMovie module

1.2.8 killMS.MergeSols module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

class killMS.MergeSols.**ClassMergeSols**(*ListFilesSols*)

Bases: object

CheckConformity()

FilterOutliers(*Sigma*)

MakeDicoMerged()

NormMatrices(*G*)

Save(*FileOut*)

SortInFreq()

killMS.MergeSols.**GiveMAD**(*X*)

killMS.MergeSols.**driver**()

killMS.MergeSols.**main**(*options=None*)

killMS.MergeSols.**read_options**()

killMS.MergeSols.**test**()

1.2.9 killMS.PlotSols module

1.2.10 killMS.PlotSolsIm module

1.2.11 killMS.SmoothSols module

1.2.12 killMS.TestSpeedNp module

1.2.13 killMS.dsc module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.dsc.**driver**()

1.2.14 killMS.grepall module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.grepall.**driver**()

1.2.15 killMS.kMS module

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

killMS.kMS.**GiveNoise**(*options*, *DicoSelectOptions*, *IdSharedMem*, *SM*, *PM*, *PM2*, *ConfigJacobianAntenna*, *GD*)

Used to initialise KAFCA. TODO document better.

killMS.kMS.**driver**()

killMS.kMS.**main**(*OP=None*, *MSName=None*)

Docstring for main

Parameters

- **OP** – OptParse object describing killMS options.
- **MSName** – Data file name to work on.

killMS.kMS.**read_options**()

This function reads the already-initialised default parset object. It then redefines all options as part of an optparse call. After this, it reads the command-line inputs for all options; defaults are kept when not provided via commandline. Finally, it calls on killMS.Other.MyPickle to save the actual parset call for this run. It returns the optparse object for this run.

defines a new MyOptParse object. depends on ReadCFG, ClassPrint, Other.ModColor, Other.logo

Return type

<module 'killMS.Parset.MyOptParse' from '/home/bonnassieux/Wirtinger/killMS/killMS/Parset/MyOptParse.py'>

1.2.16 Module contents

killMS, a package for calibration in radio interferometry. Copyright (C) 2013-2017 Cyril Tasse, l'Observatoire de Paris, SKA South Africa, Rhodes University

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

PYTHON MODULE INDEX

k

t

INDEX

E

F

G

H

I

J

K

L

R

S

T

U

W

Z